

The Hybrid Meta-Scheduler

SoC '26 KernelMind | Assignment 3 Report

IIT Bombay

Overview

This report documents the design, implementation, and evaluation of a Reinforcement Learning-driven CPU Meta-Scheduler. Rather than implementing a single static scheduling policy, the agent learns to *select* among four baseline scheduling algorithms at each episode, choosing the policy best suited to the observed workload state. The system is modelled as a tabular Markov Decision Process (MDP) and solved using Q-Learning with ϵ -greedy exploration.

Part 1: Architecture and Baseline Simulators

Process and Workload Representation

Each process is represented as an object carrying: `arrival_time`, `burst_time`, `remaining_time`, `start_time`, `finish_time`, `has_started`, and `is_over`. The workload generator produces 10 processes per episode with burst times sampled uniformly from $[1, 45]$ ticks and arrival times sorted-uniformly from $[0, 100]$ ticks.

Baseline Schedulers

Four baseline scheduling algorithms were implemented, each receiving the workload list and returning a metrics tuple:

- **FCFS:** Executes processes strictly in arrival order with no preemption.
- **SJF (non-preemptive):** At each decision point, selects the shortest burst job from all arrived processes.
- **Round Robin (RR, $q = 4$):** Cycles through arrived processes giving each a fixed 4-tick time slice before yielding.
- **Random Agent:** At each decision point, selects uniformly at random from arrived processes. Serves as the absolute performance floor.

Design Question 1: The Preemption Paradigm

Preemption is mathematically necessary to achieve the absolute minimum mean wait time because non-preemptive schedulers cannot react to newly arrived short jobs while a long job occupies the CPU. The optimal preemptive analogue of SJF is **Shortest Remaining Time First (SRTF)**: whenever a new job arrives whose remaining burst is shorter than the current job's remaining time, the CPU immediately preempts and switches.

Consider the following failure case in non-preemptive SJF. A large database backup of burst time $B = 500$ ticks begins at time $t = 0$. One tick later, at $t = 1$, twelve tiny web requests arrive each with burst $b = 2$ ticks. Under non-preemptive SJF, the scheduler is locked into the backup until $t = 500$. Each web request then waits approximately 499 ticks, yielding a mean wait ≈ 499 ticks for all 12 requests. Under SRTF, at $t = 1$ the backup (remaining = 499) is preempted and the 12 tiny requests are served immediately; they complete by $t = 1 + 24 = 25$ with a mean wait of only ~ 12 ticks. Non-preemptive SJF thus produces

catastrophically high tail latencies whenever a long job begins just before a burst of short arrivals. The inability to revoke the CPU mid-burst is the fundamental flaw.

Part 2: Evaluation Metrics

All four scheduler functions return a uniform metrics tuple:

Metric	Definition
Mean Wait Time	$\bar{W} = \frac{1}{n} \sum_{i=1}^n (S_i - A_i)$, where S_i is start time and A_i is arrival time. Primary throughput indicator.
P90 Wait Time	The wait time at the 90th percentile of the sorted wait distribution. Exposes worst-case latency experienced by the slowest 10% of jobs.
Mean Turnaround	$\bar{T} = \frac{1}{n} \sum_{i=1}^n (F_i - A_i)$, where F_i is finish time. Total time a process spends in the system.
Mean Response Time	$\bar{R} = \frac{1}{n} \sum_{i=1}^n (S_i - A_i)$ for non-preemptive; for RR, S_i is the first-scheduled time.
Jain's Fairness Index	$\mathcal{J} = \frac{(\sum_{i=1}^n w_i)^2}{n \cdot \sum_{i=1}^n w_i^2} \in [\frac{1}{n}, 1]$. A value of 1.0 indicates perfectly equal wait times.

Part 3: MDP Environment Formulation

State Space

The state of the scheduler is characterised by two features of the incoming workload:

- **Average arrival time bin:** the mean arrival time of the 10 processes, bucketed into three bins $[0, 30)$, $[30, 60)$, $[60, 100]$, reflecting how spread out arrivals are across the 100-tick window.
- **Average burst time bin:** the mean burst time, bucketed as < 7 (light), $[7, 14)$ (medium), ≥ 14 (heavy).

These yield $3 \times 3 = 9$ discrete states, indexed via row-major encoding: $s = \text{arrival_bin} \times 3 + \text{burst_bin}$.

Action Space

Four discrete actions map to the four baseline schedulers: $\{0 : \text{FCFS}, 1 : \text{SJF}, 2 : \text{RR}, 3 : \text{Random}\}$.

Reward Function

Design Question 2: Reward Shaping

The final reward function is:

$$r = 30000 - 3 \bar{W}^2 - 1.1 \bar{T}^2 - 0.4 P90^2 - 2 \bar{R}^2 + 800 (\mathcal{J} - 0.5)^2$$

Why quadratic penalties, not linear? Consider a linear penalty $-c \cdot \bar{W}$. Under SJF, an agent could justify starving one large job (contributing, say, +400 to $\bar{W}$) if it simultaneously reduces the wait of many small jobs by a corresponding aggregate amount. The linear cost

is *symmetric*: 400 extra ticks on one job costs the same as 1 extra tick each on 400 jobs.

By using $-3\bar{W}^2$, the penalty grows quadratically with the wait. An agent that starves a single process causes $\bar{W}$ to increase sharply, and that increase is squared, producing a disproportionately large negative reward. Formally, for any partition of total wait $W_{\text{total}} = W_{\text{saved}} + W_{\text{starvation}}$, the quadratic penalty satisfies:

$$(W_{\text{saved}} + \Delta)^2 \gg W_{\text{saved}}^2 + \Delta^2 \quad \text{for large } \Delta,$$

making starvation of even a single job mathematically unjustifiable. The same logic applies to the quadratic penalties on turnaround, P90, and response time.

The term $+800(\mathcal{J} - 0.5)^2$ rewards any strong deviation of the fairness index from the neutral midpoint of 0.5, incentivising either very high fairness or penalising near-neutral distributions. The base constant 30000 keeps rewards positive across all encountered workloads.

Part 4: The Q-Learning Agent

The agent maintains a Q-table $Q \in \mathbb{R}^{9 \times 4}$ (9 states, 4 actions) initialised to zero. At each episode:

1. Observe state s from `env.reset()`.
2. Select action a via ε -greedy: with probability ε choose uniformly at random; otherwise $a = \arg \max_{a'} Q(s, a')$.
3. Execute a , receive reward r and next state s' from `env.step(a)`.
4. Apply the Bellman update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

5. Decay ε : $\varepsilon \leftarrow \varepsilon \cdot 0.9995$, with a hard cutoff at $\varepsilon_{\min} = 0.0001$ beyond which ε is set exactly to 0 to enforce pure exploitation.

Hyperparameters used: $\alpha = 0.1$, $\gamma = 0.9$, $\varepsilon_0 = 1.0$, $\varepsilon_{\min} = 0.0001$, decay rate = 0.9995.

Part 5: Training and Results

Training ran for 30,000 episodes. Baselines were evaluated over 300 independent workloads each. The convergence plot (mean wait time, 500-episode moving average) demonstrated a clean descent from ≈ 65 ticks at the start of training to stable convergence near 46 ticks, closely matching the SJF baseline.

Comparative Metrics Table

Algorithm	Mean Wait	P90 Wait	Mean TAT	JFI	Mean Resp
FCFS	65.18	116.53	88.50	0.68	65.18
SJF	46.40	106.85	69.64	0.48	46.40
RR	93.05	141.19	116.00	0.76	6.15
Random	66.04	134.38	89.43	0.58	66.04
RL Agent	45.67	103.97	68.57	0.48	45.67

TAT = Turnaround Time. JFI = Jain's Fairness Index. All times in ticks.

Design Question 3: Output Analysis

Did the agent form a hybrid policy? In practice the trained agent converged almost

exclusively to SJF (300 out of 300 greedy evaluation episodes), producing an action distribution of FCFS: 0, SJF: 300, RR: 0, Random: 0. This is not a failure of learning; it is the *correct* policy given the reward structure.

Comparison with SJF and RR. The RL Agent's mean wait of 45.67 ticks marginally *outperforms* pure SJF (46.40 ticks) due to workload variance across independent evaluation episodes; over a large enough sample the two converge. On every other metric the agent matches SJF closely. Round Robin achieves the best response time (6.15 ticks) and highest fairness (JFI = 0.76) because it preempts aggressively, but its mean wait (93.05) and turnaround (116.00) are nearly double those of SJF due to context-switching overhead with a $q = 4$ quantum relative to the burst distribution.

Why SJF dominates under this reward function. The reward function applies the largest coefficient to $\bar{W}^2$ (coefficient 3) and $\bar{T}^2$ (coefficient 1.1). SJF is the provably optimal non-preemptive algorithm for both metrics simultaneously. Although the fairness term $+800(\mathcal{J} - 0.5)^2$ could in principle reward RR's higher JFI, the numerical contribution is small: for SJF, $800 \times (0.48 - 0.5)^2 \approx 0.32$, while the wait-time penalty difference between RR and SJF is on the order of $3 \times (93^2 - 46^2) \approx 19,000$. The fairness incentive is orders of magnitude weaker than the throughput penalty, so the Q-table correctly assigns dominant value to action 1 (SJF) in all encountered states. An agent that selects SJF is not exhibiting degenerate behaviour; it has learned the globally optimal policy for the given objective.

Conclusion

The Meta-Scheduler successfully learns a stable scheduling policy through tabular Q-Learning over 30,000 episodes. The convergence plot clearly demonstrates the agent surpassing FCFS, Random, and RR baselines, eventually matching and marginally exceeding SJF performance. The quadratic reward structure effectively prevents starvation by penalising large wait times disproportionately. The resulting policy of exclusively selecting SJF is mathematically rational: given the reward coefficients, no alternative action produces a higher expected return. To elicit a genuinely hybrid policy, future work could increase the fairness coefficient substantially or introduce a context-switch penalty to raise the cost of Round Robin preemptions, forcing the agent to balance throughput against fairness rather than optimising one objective alone.